

Boolean Algebra

Originally designed for philosophical statements about “truth” and “falsehood”, or whether a conclusion is valid given a set of premises.

Boolean Algebra

“If A then B” very quickly becomes complex for “If
A OR B but not C and D ... then X”.

Boolean Algebra

Example:

If it's Tuesday or Wednesday, and it's during semester, and I'm not sick, and the trains are running or my car starts, I will go to university.

Boolean Algebra

Example:

If it's (Tuesday or Wednesday), and it's (during semester), and (I'm not sick), and (the trains are running or my car starts), I will go to university.

Boolean Algebra

We can assign algebraic variables to those states,
and combine them mathematically:

Tuesday	Wednesday	Semester	Sick	Train	Car
A	B	C	D	E	F

Boolean Algebra

If it's (Tuesday or Wednesday), and it's (during semester), and (I'm not sick), and (the trains are running or my car starts), I will go to university.

If (A or B), and (C), and (NOT D), and (E or F), I will go to university.

What about binary??

A, B, C etc are our variables here, but their value changes depending on conditions like any other algebra. So now we can say if a thing is true then we give it the value 1, otherwise we give it the value 0. A “Boolean” can only be true or false, which maps well to binary 1 and 0.

What about binary??

So on Tuesday ($A=1$, $B=0$), during semester ($C=1$), when I'm not sick ($D = 0$) and the trains are running ($E=1$, $F=0$, my car needs a new fusible link ;)

If (1 or 0), and (1), and (NOT 0), and (1 or 0), I will go to university.

How does this help?

One of the things Boole did was define “truth tables” for every possible combination of 2 inputs (and by extension every possible combination, because the result of 2 inputs is 1 output, so you can always have 2 inputs).

Truth tables

Here is one truth table, for “A or B”. You can see that the result is true when A is true, or B is true (or both).

A	B	Result
0	0	0
0	1	1
1	0	1
1	1	1

Truth tables

Let's “collapse” (or “evaluate”) the OR expressions here:
If (1 or 0), and (1), and (NOT 0), and (1 or 0), I will go to university.

“1 OR 0” becomes 1 – A is true, so the whole thing is true

Similarly, my car is (often) broken, but the train is running
– “1 OR 0” == 1

Truthiness

Where are we now?

If (1) and (1) and (NOT 0) and (1) I will go to university.
NOT 0 is straightforward. A Boolean can only have 2 values – 0 or 1. So if it's NOT 0, it must be 1:

- If (1) and (1) and (1) and (1) I will go to university.

Truth table redux

Now all we have is AND. An AND relationship is true if the two inputs are both true

A	B	Result
0	0	0
0	1	0
1	0	0
1	1	1

It's a University day!

- If (1) and (1) and (1) and (1) I will go to university.

Doesn't matter where we start – any 2 choices above give “1 AND 1” which is 1. So we end up with:

If 1 I will go to university.

1 is by definition true, so I am going to university.

It's a University day!

Usually we would also assign a variable to “going to university”:

- If (A or B), and (C), and (NOT D), and (E or F),
I will go to university.
- $U = (A \text{ or } B) \text{ and } (C) \text{ and } (\text{NOT } D) \text{ and } (E \text{ or } F)$
 - $U = 1$, so here we are

What about computers?

Computers (and symbolic logic) are all about manipulating 1s and 0s to get a result. But the combination of 1s and 0s sometimes gives different results depending on what we want to achieve.

What even Boolean

For example, combining 1 and 0 by adding them should give 1.

Combining 1 and 0 by multiplying should give 0.

Similarly, $1 \text{ plus } 1 = 2$ (which is a problem in binary) but $1 \text{ times } 1 = 1$. Again, different contexts, different results from a combination.

Boolean computing

Since computers don't understand context, or what we are trying to achieve with different things, we need some way of formally describing a process so that we can then build the electronics to make it happen.

Boolean computing

Boolean algebra already has operators to say
"In this context, combining 1 with 0 results in 1"
or "In this context combining 1 with 0 results in
0".

Inputs and outputs

When we are combining 1s and 0s, it's not very useful to use the value all the time, and again what we want is general rules. So for addition, we're really looking at “what happens if we add A and B?”

Gates

All we have to do then is build a circuit to give us that result electronically. Of course, we could have skipped the Boolean algebra and just built a circuit to do this and called it a “multiplier”. But the underlying logic is so useful, we call it an AND “gate”, and they are used everywhere – not just for multiplying things.

Gates

For 2 possible inputs, given that each input has 2 possible states (because binary), there are in fact 16 possible truth tables. You can prove that yourself, or you can go look at the nand2tetris website – link is in the resources for this unit.

So in theory there are 16 different Boolean operators, and hence “gates” or circuits.

Gates

In practice, some of those states are rarely used, and some are very common, The most common ones are probably AND, OR, and XOR (and their opposites – NAND, NOR and XNOR)

Adding!

I'm only going to look at one more truth table here – the XOR gate. You can think about how it might help add things.

A	B	Result
0	0	0
0	1	1
1	0	1
1	1	0

Adding!

“OR” is the traditional “If A is 1 or B is 1 then the result is 1”. But XOR – exclusive OR – is the answer, for example, to “Would you like tea or coffee”? You can't have both!